Circuit Breaker

eason: 4 · Enterprise · Category: Enterprise · Level: Intermediate · Build with Waqa

Today's Hook

- Today: Circuit Breaker as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Circuit Breaker structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Circuit Breaker
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/enterprise/circuitbreaker/CircuitBreaker.java
package com.buildwithwaqar.patterns.enterprise.circuitbreaker;

import java.time.Duration;
import java.time.Instant;
import java.util.concurrent.Callable;
import java.util.function.Supplier;

public final class CircuitBreaker {
    public enum State { CLOSED, OPEN, HALF_OPEN }

    private final int failureThreshold;
    private final Duration openTimeout;
    private final Supplier<Instant> clock;

    private State state = State.CLOSED;
    private int failures;
    private Instant openedAt = Instant.EPOCH;

    public CircuitBreaker(int failureThreshold, Duration openTimeout, Supplier<Instant> clock) {
        this.failureThreshold = failureThreshold;
        this.openTimeout = openTimeout;
        this.clock = clock;
    }
```

Implementation (2)

```
    public State state() { return state; }

    public <T> T call(Callable<T> action) throws Exception {
        if (state == State.OPEN) {
            if (clock.get().isAfter(openedAt.plus(openTimeout))) {
                state = State.HALF_OPEN;
            } else {
                throw new IllegalStateException("circuit open");
            }
        }
        try {
            var result = action.call();
            failures = 0;
            state = State.CLOSED;
            return result;
        } catch (Exception e) {
            failures++;
            if (failures >= failureThreshold || state == State.HALF_OPEN) {
                state = State.OPEN;
                openedAt = clock.get();
            }
```

Implementation (3)

```
                throw e;
            }
        }
    }
```

Demo.main

```
package com.buildwithwaqar.patterns.enterprise.circuitbreaker;
import java.time.*;
import java.util.concurrent.atomic.AtomicReference;
public final class Demo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Circuit Breaker walkthrough ===");
    var now = new AtomicReference<>(Instant.parse("2026-01-01T00:00:00Z"));
    var cb = new CircuitBreaker(2, Duration.ofSeconds(30), now::get);
    for (int i = 0; i < 2; i++) {
      try { cb.call(() -> { throw new RuntimeException("down"); }); }
      catch (Exception e) { System.out.println("fail" + i + " state=" + cb.state()); }
    }
    try { cb.call(() -> "ok"); } catch (Exception e) { System.out.println("shortCircuit=" + e.getMessage()); }
    now.set(now.get().plusSeconds(31));
    System.out.println("afterTimeout=" + cb.call(() -> "ok") + " state=" + cb.state());
  }
}
```

Walkthrough Output

```
=== Circuit Breaker walkthrough ===
fail0 state=CLOSED
fail1 state=OPEN
shortCircuit=circuit open
afterTimeout=ok state=CLOSED
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Circuit Breaker
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Circuit Breaker → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.